

Contents

I	Creación de preguntas de opción múltiple	2
1	Estructura del paquete calcprop-quiz	3
2	Función auxiliar CuestionesJuntas	4
3	Clase Marcador	5
3.1	Implementación	5
3.1.1	Función auxiliar Siguiente (x, y)	6
4	Las clases Supuesto, Cuestion, ProblemaTipo y ProblemaTipoProfe	7
4.1	Clase Supuesto(enunciado, semantica, precondition)	7
4.2	Clase Cuestion(enunciado, semantica, precondition)	7
4.3	Clase ProblemaTipo	7
4.3.1	Implementación	8
4.4	Ejemplo de uso	10
4.5	Clase ProblemaTipoProfe	10
5	Clase ProblemaVF	12
5.1	Ejemplo de uso	12

Part I

Creación de preguntas de opción múltiple

Chapter 1

Estructura del paquete calcprop-quiz

```
from calcprop import *
```

```
«Aparato auxiliar para la creación de problemas de opción múltiple»
```

```
«Metodo auxiliar para juntar cuestiones en formato para profe»
```

```
«Definición de la clase Marcador»
```

```
«Definición de la clase Supuesto»
```

```
«Definición de la clase Cuestion»
```

```
«Definición de la clase ProblemaTipo»
```

```
«Definición de la clase ProblemaTipoProfe»
```

```
«Definición de la clase ProblemaVF»
```

Chapter 2

Función auxiliar CuestionesJuntas

La función `CuestionesJuntas` toma una lista de componentes del tipo usado en `ProblemaTipo` y “junta” (aplana) las sublistas de `Question`-es en la lista principal. Esto permite que `ProblemaTipoProfe` muestre todas las cuestiones posibles para cada enunciado en lugar de una sola.

```
def CuestionesJuntas(l):  
    def CreaLista(t):  
        return t if isinstance(t, list) else [t]  
    p = []  
    for e in l:  
        if isinstance(e, str):  
            p.append(e)  
        elif not isinstance(CreaLista(e)[0], Question):  
            p.append(e)  
        else:  
            p.extend(CreaLista(e))  
    return p
```

Chapter 3

Clase Marcador

Usaremos la clase `Marcador` como una herramienta auxiliar para construir el árbol con todas las combinaciones posibles entre una conjunto de enunciados y un conjunto de cuestiones. Posteriormente se descartarán de dicho árbol las combinaciones de enunciados y cuestiones que no tengan solución.

Clase `Marcador`

- `objetivo` es una lista de números naturales mayores que cero

La clase `Marcador` es un iterador. Si `objetivo=[n1,...,nk]` genera todas las listas de la forma `[m1,...,mk]` tales que cada `mi` es un número natural menor que `ni`

```
for i in Marcador([2,3,1]):  
    print(i)
```

```
[0, 0, 0]  
[0, 1, 0]  
[0, 2, 0]  
[1, 0, 0]  
[1, 1, 0]  
[1, 2, 0]
```

3.1 Implementación

El atributo `self.data` es la lista de números que hemos dado como argumento. En el ejemplo anterior, `[2,3,1]`.

Para construir un iterador necesitamos los métodos `__iter__` y `__next__`.

```
class Marcador:  
    def __init__(self, data):  
        self.data = data  
    def __iter__(self):  
        self.p = [0 for x in self.data]  
        return self  
    def __next__(self):  
        def Siguiente(x,y):  
            if x == [] :  
                return []  
            s = Siguiente(x[1:],y[1:])  
            if s == []:  
                if x[0]+1 == y[0]:  
                    return []  
                else:  
                    return [x[0]+1] + [0 for i in x[1:]]  
            else:  
                return [x[0]] + s  
        if self.p == []:  
            raise StopIteration  
        n = self.p  
        self.p = Siguiente(self.p, self.data)  
        return n
```

El atributo `self.p` contiene la lista que entregará `__next__` en la siguiente invocación al método; y el método `__iter__` genera la semilla de dicha lista, en particular dicha semilla es una lista de ceros.

```
def __iter__(self):
    self.p = [0 for x in self.data]
    return self
```

`__next__` devuelve `self.p` en su estado actual (`n`), y coloca en `self.p` la lista siguiente. Si `self.p` es vacía detiene la iteración.

```
def __next__(self):
    «Definición de la función local Siguiente»
    if self.p == []:
        raise StopIteration
    n = self.p
    self.p = Siguiente(self.p, self.data)
    return n
```

Dentro del método `__next__` definimos de manera recursiva la función auxiliar local `S`, que calcula la lista siguiente.

3.1.1 Función auxiliar Siguiente (`x`, `y`)

- `x`: lista de números naturales `[m_1...m_r]` tales que `m_i < n_i` (es la lista de la que queremos calcular la siguiente).
- `y`: lista de números naturales `[n_1...n_r]` mayores que cero (lista de topes).

Devuelve `[k_1...k_r]` la siguiente lista a `[m_1...m_r]` (según el orden lexicográfico) tal que `k_i < n_i`. En caso de no existir tal lista devuelve `[]`.

```
def Siguiente(x,y):
    if x == [] :
        return []
    s = Siguiente(x[1:],y[1:])
    if s == []:
        if x[0]+1 == y[0]:
            return []
        else:
            return [x[0]+1] + [0 for i in x[1:]]
    else:
        return [x[0]] + s
```

Chapter 4

Las clases Supuesto, Cuestion, ProblemaTipo y ProblemaTipoProfe

4.1 Clase Supuesto(enunciado, semantica, precondition)

Clase Supuesto(enunciado, semantica, precondition)

Es un constructor que sirve para almacenar tres aspectos (atributos) del supuesto de un problema.

enunciado string que presenta la supuesto en lenguaje humano.

semantica proposición que traduce parcialmente el significado lógico del **enunciado** (contiene lo necesario para deducir las respuestas).

precond es una proposición o bien cualquiera de los valores True, False. El Supuesto será incluido en el problema solo si la evaluación de **precond** es True.

```
class Supuesto:
    def __init__(self, enunciado, semantica, precondition=True):
        self.e = enunciado
        self.s = semantica
        self.p = precondition
```

4.2 Clase Cuestion(enunciado, semantica, precondition)

Clase Cuestion(enunciado, semantica, precondition)

Es un constructor que sirve para almacenar tres aspectos (atributos) de una cuestión de un problema.

enunciado string que presenta la cuestión en lenguaje humano.

semantica proposición P tal que test(P, semántica del conjunto de supuestos) devuelve la respuesta correcta.

precond es una proposición o bien cualquiera de los valores True, False. La Cuestion será incluida en el problema solo si la evaluación de **precond** es True.

```
class Cuestion:
    def __init__(self, enunciado, semantica, precondition=True, exp=""):
        self.e = enunciado
        self.s = semantica
        self.p = precondition
        self.x = exp
```

4.3 Clase ProblemaTipo

Clase ProblemaTipo (supuestos_y_cuestiones)

supuestos_y_cuestiones es una lista de listas de strings, Supuesto-s, o Cuestion-es.

Cada lista representa un conjunto de opciones excluyentes. Para mayor comodidad, en el caso de que una de esas listas consista en un único objeto (una única opción), también se permite escribir directamente el objeto sin encerrarlo en una lista, es decir, la lista `supuestos_y_cuestiones` también admite objetos de tipo: `string`, `Supuesto` o `Cuestion`.

4.3.1 Implementación

La clase `ProblemaTipo` es un iterador que genera todas las versiones aceptables de un problema. Cada variante resulta de tomar una de las opciones de cada una de las listas en `supuestos_y_cuestiones`, pero si la precondición de alguna de las opciones elegidas es `False` la correspondiente variante del problema es rechazada.

- El atributo `self.e` contiene la lista `supuestos_y_cuestiones`. Cada elemento de `supuestos_y_cuestiones` es a su vez una lista de opciones (excluyentes) correspondiente a cada componente (cada parte) del texto del problema. La combinación de distintas opciones crea el texto completo de una variante del problema.

El método `__iter__` inicializa el iterador. El método `__next__` obtiene la siguiente variante del problema, pero asegurándose de que las precondiciones son satisfechas (si no la variante es desechada).

```
class ProblemaTipo:
    def __init__(self, supuestos_y_cuestiones):
        self.e = supuestos_y_cuestiones

    def __iter__(self):
        self.l = [x if isinstance(x,list) else [x] for x in self.e]
        self.long = len(self.l)
        self.i = iter(Marcador([len(x) for x in self.l]))
        self.c = 0
        return self

    def __next__(self):
        self.c += 1
        while True:
            try:
                variante = next(self.i)
            except StopIteration:
                raise StopIteration

            enunciado = ""
            hipotesis = []
            cuestiones = []

            for n in range(self.long+1):
                if n == self.long:
                    return (str(self.c), enunciado, cuestiones)

                componente = self.l[n][variante[n]]
                if isinstance(componente, str):
                    enunciado = enunciado + componente

                elif isinstance(componente, Supuesto):
                    if test(componente.p, hipotesis):
                        enunciado = enunciado + componente.e
                        hipotesis = hipotesis + [componente.s]
                    else:
                        print('\n Supuesto: ' + str(componente.e) \
                              + ' rechazado por ' + str(componente.p) + '\n')
                        break

                elif isinstance(componente, Cuestion):
                    if test(componente.p, hipotesis):
                        cuestiones = cuestiones + \
                            [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
                    else:
                        cuestiones = cuestiones + \
                            [(componente.e, 'rechazada por ' + str(componente.p), 0, componente.x)]
                        print('\n Cuestion: ' + str(componente.e) \
                              + ' rechazada por ' + str(componente.p) + '\n')
                        break
```

El atributo `self.l` es la lista `self.e` normalizada para que todos sus objetos sean listas; `self.long` es el número de componentes que constituyen el texto de cada variante. `self.i` es el iterador y `self.c` un contador.

```
def __iter__(self):
    self.l = [x if isinstance(x,list) else [x] for x in self.e]
    self.long = len(self.l)
    self.i = iter(Marcador([len(x) for x in self.l]))
    self.c = 0
    return self
```

La generación de **variantes** de un **ProblemaTipo** se produce dentro de un bucle que solo se detiene cuando el intento (**try**) de iterar una vez más, **next(self.i)** falla por haberse agotado todas las variantes.

self.long guarda el número de elementos (es decir, de cadenas, **Supuesto-s** y **Cuestion-es**) que constituyen el texto del problema.

En cada iteración 1) **enunciado**, **hipotesis** y **cuestiones** comienzan estando vacías; y 2) se recorren uno a uno los componentes de la variante que se intenta construir.

Una vez se han recorrido todos los componentes se devuelve la lista: (**str(self.c)**, **enunciado**, **cuestiones**) que constituye la variante concreta del problema que se ha generado en esa iteración. Cada tipo de **componente** es tratado de una forma distinta.

```
def __next__(self):
    self.c += 1
    while True:
        try:
            variante = next(self.i)
        except StopIteration:
            raise StopIteration

        enunciado = ""
        hipotesis = []
        cuestiones = []

        for n in range(self.long+1):
            if n == self.long:
                return (str(self.c), enunciado, cuestiones)

            componente = self.l[n][variante[n]]
            if isinstance(componente, str):
                enunciado = enunciado + componente

            elif isinstance(componente, Supuesto):
                if test(componente.p, hipotesis):
                    enunciado = enunciado + componente.e
                    hipotesis = hipotesis + [componente.s]
                else:
                    print('\n Supuesto: ' + str(componente.e) \
                        + ' rechazado por ' + str(componente.p) + '\n')
                    break

            elif isinstance(componente, Cuestion):
                if test(componente.p, hipotesis):
                    cuestiones = cuestiones + \
                        [(componente.e,(True if test(componente.s, hipotesis) else False),1,componente.x)]
                else:
                    cuestiones = cuestiones + \
                        [(componente.e,'rechazada por ' + str(componente.p),0,componente.x)]
                    print('\n Cuestion: ' + str(componente.e) \
                        + ' rechazada por ' + str(componente.p) + '\n')
                    break
```

- Cuando el **componente** es una cadena, se considera dicha **componente** es parte del **enunciado** del problema, por lo que consecuentemente es añadido al mismo (es decir, se concatena al string **enunciado**).
- Cuando el **componente** es un **Supuesto**, se testea la **precondicion**. Si **componente.p** es **True** o no es refutable a partir de las **hipotesis** entonces **componente.e** es añadido al **enunciado** y la proposición **componente.s** es añadida a **hipotesis**. En caso contrario la variante del problema es completamente descartada y se imprime en la pantalla el motivo.
- Cuando el **componente** es una **Cuestion**, se testea la **precondicion**. Si **componente.p** es **True** o no es refutable a partir de las **hipotesis** entonces el par (**componente.e**, **True**!=**False** dependiendo de lo que corresponda) es añadido a la lista de **cuestiones**. En caso contrario la variante del problema es completamente descartada y se imprime en la pantalla el motivo.

```

if isinstance(componente, str):
    enunciado = enunciado + componente

elif isinstance(componente, Supuesto):
    if test(componente.p, hipotesis):
        enunciado = enunciado + componente.e
        hipotesis = hipotesis + [componente.s]
    else:
        print('\n Supuesto: ' + str(componente.e) \
              + ' rechazado por ' + str(componente.p) + '\n')
        break

elif isinstance(componente, Cuestion):
    if test(componente.p, hipotesis):
        cuestiones = cuestiones + \
            [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
    else:
        cuestiones = cuestiones + \
            [(componente.e, 'rechazada por ' + str(componente.p), 0, componente.x)]
        print('\n Cuestion: ' + str(componente.e) \
              + ' rechazada por ' + str(componente.p) + '\n')
        break

```

4.4 Ejemplo de uso

```

p = ProblemaTipo( \
    [
        "Considere ",
        [
            Supuesto("A ", v("A")),
            Supuesto("B ", v("B")),
        ],
        [
            Supuesto("y que A implica C. ", v("A") >> v("C")),
            Supuesto("y que B equivale a D. ", v("B") ** v("D")),
        ],
        "Conteste: ",
        [
            Cuestion("¿Se da C?", v("C")),
            Cuestion("¿Se da D?", v("D")),
            Cuestion("¿Se da E?", v("E"), v("D")),
        ],
    ],
)

for i in p:
    print(i)

```

```

('1', 'Considere A y que A implica C. Conteste: ', [(('¿Se da C?', True, 1, '')])
('2', 'Considere A y que A implica C. Conteste: ', [(('¿Se da D?', False, 1, '')])

```

```

Question: ¿Se da E? rechazada por v('D')

```

```

('3', 'Considere A y que B equivale a D. Conteste: ', [(('¿Se da C?', False, 1, '')])
('4', 'Considere A y que B equivale a D. Conteste: ', [(('¿Se da D?', False, 1, '')])

```

```

Question: ¿Se da E? rechazada por v('D')

```

```

('5', 'Considere B y que A implica C. Conteste: ', [(('¿Se da C?', False, 1, '')])
('6', 'Considere B y que A implica C. Conteste: ', [(('¿Se da D?', False, 1, '')])

```

```

Question: ¿Se da E? rechazada por v('D')

```

```

('7', 'Considere B y que B equivale a D. Conteste: ', [(('¿Se da C?', False, 1, '')])
('8', 'Considere B y que B equivale a D. Conteste: ', [(('¿Se da D?', True, 1, '')])
('9', 'Considere B y que B equivale a D. Conteste: ', [(('¿Se da E?', False, 1, '')])

```

4.5 Clase ProblemaTipoProfe

Clase ProblemaTipoProfe (supuestos_y_cuestiones)

`supuestos_y_cuestiones` es una lista de listas de strings, `Supuesto`-s, o `Cuestion`-es.

Para que sea más fácil la revisión y mantenimiento de cada pregunta, esta versión muestra todas las cuestiones posibles para cada enunciado. También muestra las cuestiones excluidas y el motivo de la exclusión. Usa `CuestionesJuntas` para aplanar las sublistas de cuestiones antes de iterar. Tan solo cambia el tratamiento de las cuestiones en el método `__next__`:

```
class ProblemaTipoProfe:
    def __init__(self, supuestos_y_cuestiones):
        self.e = CuestionesJuntas(supuestos_y_cuestiones)

    «Método __iter__ para la clase ProblemaTipo»
    «Método __next__ para la clase ProblemaTipoProfe»
```

Por tanto este trozo del código es igual que para el `ProblemaTipo` salvo la última parte

```
def __next__(self):
    self.c += 1
    while True:
        try:
            variante = next(self.i)
        except StopIteration:
            raise StopIteration

        enunciado = ""
        hipotesis = []
        cuestiones = []

        for n in range(self.long+1):
            if n == self.long:
                return (str(self.c), enunciado, cuestiones)

        componente = self.l[n][variante[n]]
        if isinstance(componente, str):
            enunciado = enunciado + componente

        elif isinstance(componente, Supuesto):
            if test(componente.p, hipotesis):
                enunciado = enunciado + componente.e
                hipotesis = hipotesis + [componente.s]
            else:
                print('\n Supuesto: ' + str(componente.e) \
                    + ' rechazado por ' + str(componente.p) + '\n')
                break

        elif isinstance(componente, Cuestion):
            if test(componente.p, hipotesis):
                cuestiones = cuestiones + \
                    [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
            else:
                cuestiones = cuestiones + \
                    [(componente.e, 'rechazada por ' + str(componente.p), 0)]
```

que cambia en su última parte

```
if isinstance(componente, str):
    enunciado = enunciado + componente

elif isinstance(componente, Supuesto):
    if test(componente.p, hipotesis):
        enunciado = enunciado + componente.e
        hipotesis = hipotesis + [componente.s]
    else:
        print('\n Supuesto: ' + str(componente.e) \
            + ' rechazado por ' + str(componente.p) + '\n')
        break

elif isinstance(componente, Cuestion):
    if test(componente.p, hipotesis):
        cuestiones = cuestiones + \
            [(componente.e, (True if test(componente.s, hipotesis) else False), 1, componente.x)]
    else:
        cuestiones = cuestiones + \
            [(componente.e, 'rechazada por ' + str(componente.p), 0)]
```

Chapter 5

Clase ProblemaVF

Para crear cuestiones del tipo Verdadero/Falso.

```
from random import sample
class ProblemaVF():
    def __init__(self, enunciado, cuestiones, NumPreguntas):
        self.e = enunciado
        self.c = cuestiones
        self.NumPreguntas = NumPreguntas

    def __iter__(self):
        self.contador = 0
        return self

    def __next__(self):
        cuestiones = sample(self.c, self.NumPreguntas)
        self.contador += 1
        return (str(self.contador), self.e, cuestiones)
```

5.1 Ejemplo de uso

Cuatro exámenes con dos preguntas tipo test cada uno.

```
enunciado = "Marque las verdaderas:"

banco = [
    ("Todo alumno que va a clase aprueba", False),
    ("Todo alumno que suspende va a clase", False),
    ("Todo alumno que sabe aprueba", True),
    ("Si aprueban todos eres buen profesor", False),
    ("Si suspenden todos eres mal profesor", False),
    ("Si suspenden todos es frustrante", True),]

p = ProblemaVF (enunciado, banco, 3)      # tres preguntas por variante
p0 = iter(p)

for i in range(4):                        # cuatro variantes
    print(next(p0))
```

```
('1', 'Marque las verdaderas:', [('Si suspenden todos es frustrante', True), ('Todo alumno que suspe
('2', 'Marque las verdaderas:', [('Todo alumno que sabe aprueba', True), ('Si aprueban todos eres bu
('3', 'Marque las verdaderas:', [('Si aprueban todos eres buen profesor', False), ('Todo alumno que
('4', 'Marque las verdaderas:', [('Si suspenden todos es frustrante', True), ('Si aprueban todos ere
```